



Pontificia Universidad Católica Argentina

Facultad de Ingeniería y
Ciencias Agrarias
Ingeniería Informática

Trabajo Final:

Viabilidad de la aplicación del algoritmo

Shor para la Criptografía de Curva

Elíptica (ECC)

Alumno: Isoldi, Agustin Emmanuel

Tutor: Bollmann, Germán Gustavo

Índice

1	Introducción	4
2	Motivación	7
3	Objetivos	9
4	Estado del arte	10
5	Marco teórico	13
5.1	Criptografía	13
5.1.1	Criptografía Asimétrica.....	13
5.2	Criptografía de curva elíptica.....	14
5.3	Algoritmo de Shor.....	20
5.3.1	Transformada Cuántica de Fourier (QFT)	23
6	Metodología de trabajo	25
6.1	CrypTool	27
6.2	Qiskit	30
6.2.1	<i>Backend</i> utilizado: ibm_marrakesh	36
6.3	Desarrollo del Circuito Cuántico	37
7	Resultados Obtenidos.....	40
7.1	Resultados computación clásica.....	40
7.1.1	Tiempos Requeridos.....	40
7.1.2	Complejidad Computacional.....	42
7.2	Resultados Computación Cuántica	43
7.2.1	Tiempos Requeridos.....	43

7.2.2	Complejidad Computacional.....	45
7.3	Análisis y Analogía de resultados	47
7.3.1	Tiempos Requeridos.....	47
7.3.2	Complejidad Computacional.....	50
7.4	Consideraciones para tener en cuenta	51
8	Conclusión	55
9	Futuras Discusiones	57
10	Bibliografía	58
11	Anexos.....	61
11.1	Anexo 1: Qiskit Runtime Service	61
11.2	Anexo 2: OPENQASM 2.0.....	61
11.3	Anexo 3: Software ejecutable (lenguaje Python)	61
11.3.1	busqueda_curva.ipynb	61
11.3.2	shor_breaking_ecc.ipynb	62
11.3.3	ataque_ECC_fuerza_bruta_clasico.ipynb.....	62

1 Introducción

Tras el crecimiento exponencial y adopción masiva de la IA [10], una nueva tendencia tecnológica empieza a ganar terreno: la computación cuántica. A diferencia de los ordenadores clásicos, las computadoras cuánticas exploran múltiples caminos computacionales en paralelo, lo que les permite resolver problemas complejos en tiempos radicalmente menores. [10]

El desarrollo progresivo de la computación cuántica ha encendido alarmas para la seguridad informática moderna que generaron preocupación. [2] La criptografía actual, ampliamente utilizada en varios aspectos de nuestra vida cotidiana, se fundamenta en problemas matemáticos que resultan computacionalmente inviables de resolver con los recursos clásicos disponibles en el corto plazo. [1]

Sin embargo, este paradigma enfrenta una gran amenaza ante la posibilidad real de que el desarrollo de computadoras cuánticas avance de tal manera que, de manera viable, se puedan resolver dichos problemas con complejidades menores, debido al procesamiento en paralelo que brinda esta nueva tecnología. [2]

En este caso, algoritmos basados en criptografía de clave pública, como RSA (Rivest, Shamir y Adleman) y ECC (Criptografía de Curva Elíptica), dependen de la dificultad para factorizar números grandes primos en el primer caso, o resolver el logaritmo discreto elíptico en el segundo mencionado. [1] Tales tareas, según la teoría cuántica, podrían ser eficientemente más fáciles de resolver. Esta capacidad permitiría vulnerar los sistemas criptográficos tal cual conocemos hoy en día con menor complejidad de recursos y tiempos, causando un impacto global en términos de seguridad informática. [3]

Dado este escenario, la comunidad científica ha comenzado a trabajar activamente en el desarrollo de esquemas criptográficos resistentes a ataques cuánticos, agrupados bajo el término “criptografía post-cuántica”. La necesidad de migrar hacia este nuevo enfoque ya no es una posibilidad lejana, sino una urgencia tecnológica que debe ser abordada desde el presente. [3]

El principal desafío remarca que debemos mejorar nuestros sistemas de seguridad para un futuro no tan lejano. Ya sea contraatacando a la computación cuántica con algoritmos cuánticos que sean resistentes a su propia capacidad de cómputo, o encontrando alguna otra alternativa que dificulte el proceso de romper los algoritmos clásicos utilizando estas nuevas tecnologías. [3]

A continuación, presentamos un cuadro comparativo entre los ataques por fuerza bruta clásica, y estimaciones teóricas de ataques con el algoritmo de Shor a claves generadas tanto con RSA, como con ECC. [4]

Factoring algorithm (RSA)			EC discrete logarithm (ECC)			classical
n	$\approx \#$ qubits	time	n	$\approx \#$ qubits	time	time
	$2n$	$4n^3$		$f'(n)$ ($f(n)$)	$360n^3$	
512	1024	$0.54 \cdot 10^9$	110	700 (800)	$0.5 \cdot 10^9$	C
1024	2048	$4.3 \cdot 10^9$	163	1000 (1200)	$1.6 \cdot 10^9$	$C \cdot 10^8$
2048	4096	$34 \cdot 10^9$	224	1300 (1600)	$4.0 \cdot 10^9$	$C \cdot 10^{17}$
3072	6144	$120 \cdot 10^9$	256	1500 (1800)	$6.0 \cdot 10^9$	$C \cdot 10^{22}$
15360	30720	$1.5 \cdot 10^{13}$	512	2800 (3600)	$50 \cdot 10^9$	$C \cdot 10^{60}$

Imagen 1 – Comparaciones entre tiempos de ataques por el algoritmo de Shor y cantidad de qubits necesarios [4]

Como muestra la *Imagen 1*, el cuadro compara los algoritmos de RSA y ECC, muestra la equivalencia entre tamaños de clave y complejidades de resolver los ataques por ambos métodos clásicos y cuánticos. [4] Es de gran interés para la criptografía moderna profundizar sobre la capacidad que tiene la computación cuántica. La complejidad de romper las claves de RSA y ECC, conforme aumenta

el tamaño de bits, en términos de recursos (qubits), aumenta de forma polinomial con respecto al tamaño de la clave. Los tiempos necesarios también muestran este comportamiento, pero, en este caso, suma el factor del tiempo necesario en el ataque por fuerza bruta clásico; mientras de manera clásica el tiempo aumenta exponencialmente, en términos de cuántica aumenta de manera polinomial. [4]

Explicando en detalle la *Imagen 1*, proporcionado por el artículo académico que logra la estimación, n es la cantidad de bits de la clave generada por RSA o ECC, luego muestra la cantidad de qubits necesarios para que funcione el algoritmo de Shor, y, por último, el tiempo requerido para hallar la clave mediante la aplicación de este con el procesamiento de la computación cuántica. [4]

2 Motivación

La motivación para el presente trabajo radica en verificar la teoría de que los algoritmos de criptografía de clave pública, como ECC, son vulnerables ante la computación cuántica y su avance tecnológico. En definitiva, algoritmos existentes como Shor, en la teoría, tienen el potencial de atacar sistemas de seguridad de la información que se utilizan comúnmente hoy en día. [5]

En particular, buscamos replicar los valores teóricos de la dificultad computacional que implica romper una clave generada mediante criptografía de curvas elípticas utilizando técnicas clásicas frente a las ventajas potenciales que ofrece la aplicación cuántica del algoritmo de Shor. [5]

Para ello, nos motiva replicar los valores teóricos a menor escala mediante simulaciones prácticas, tanto clásicas como cuánticas, con el fin de evaluar en qué medida la computación cuántica representa una amenaza real y cuantificable para los sistemas criptográficos clásicos que se utilizan en la actualidad.

Actualmente, existen artículos teóricos, pero no se han encontrado implementaciones prácticas de acceso público que permitan probar la funcionalidad del algoritmo de Shor y evidencien sus resultados o realizar pruebas.

Puntos para tener en cuenta como parte de la motivación para el presente trabajo de carácter académico:

- Anticipar cambios e incitar nuevas medidas de seguridad. [6]
- Crear circuitos cuánticos y utilizar la capacidad brindada por los QPU (Quantum Processing Unit [7] o Unidad de Procesamiento Cuántica) reales de IBM para realizar prácticas. [8]

- Experimentar con el algoritmo de Shor y su capacidad de resolver el logaritmo discreto elíptico. [9]
- Pensar en la seguridad a largo plazo, para afrontar nuevos desafíos que implican el avance de la tecnología, nuevos algoritmos o paradigmas. [6]

3 Objetivos

El objetivo principal del presente trabajo final implica diseñar, implementar y evaluar experimentalmente un circuito cuántico como implementación funcional basado en el algoritmo de Shor para la resolución del problema del logaritmo discreto en curvas elípticas sobre instancias reducidas o de menor orden, adaptado a las restricciones del hardware actual. Comparando su desempeño en términos de complejidad temporal y espacial frente al ataque por fuerza bruta clásico, utilizando simuladores y *backends* de hardware cuántico real provistos por IBM Qiskit.

Analizar el estado del arte es fundamental para tomar la teoría que se va a llevar a la práctica clásica y cuántica. Con ello, se va a experimentar con diferentes ECC de diversa cantidad de bits de clave con el objetivo de comparar de manera práctica los ataques por fuerza bruta clásico con el circuito cuántico generado que implementa el algoritmo de Shor.

Las métricas que se evalúan comparativamente en tanto al desempeño de la ejecución clásica versus cuántica son:

- 1) Tiempo de ejecución: tiempo real del ataque.
- 2) Complejidad computacional: comparación entre el proceso teórico planteado por Shor vs los valores obtenidos del experimento.

Al finalizar, con la comparación de ambos resultados obtenidos, poder concluir si con el circuito implementado de Shor en una computación cuántica real es comparable con la complejidad teórica estimada por Shor en 1994 cuando no existía esta computadora.

4 Estado del arte

El presente trabajo toma como pilar la resolución del problema del logaritmo elíptico a partir de programación clásica y cuántica con el objetivo de resolver una clave generada por algoritmos de curvas elípticas. En este trabajo no se entrará en detalle sobre la teoría de curvas elípticas, ya que no aporta valor al objetivo de esta investigación. Sin embargo, se abordará los principios aplicados a los criptosistemas de curvas elípticas, con el objetivo de dar un contexto y explicación simple que demuestre como se produce la generación de claves. Mostrando la dificultad que implica obtener estas claves, tanto de manera clásica, como cuántica, para un tercero que tenga intenciones de comprometer la confidencialidad del mensaje. [5]

En el artículo académico llamado “*Mathematics of Computation*”, Koblitz, N. [11] publicado en 1987 explica de manera concisa la idea de la criptografía de curva elíptica. El propósito central radica en el uso del grupo de puntos generado por la ecuación de una función que representa una curva elíptica sobre un cuerpo finito para construir un criptosistema de clave asimétrica o pública. Si bien al momento de la publicación ya existía la criptografía de clave pública, con algoritmos como Diffie-Hellman (problema del logaritmo discreto) o RSA (factorización de números grandes), Koblitz [11] llevó el problema del logaritmo discreto al grupo matemático definido por curvas elípticas. Más adelante, explicaremos en detalle cómo funciona la idea general de la criptografía de curvas elípticas y cuál es su principal característica.

Peter Shor en 1994 publica teóricamente un algoritmo capaz de factorizar números enteros y de resolver el problema del logaritmo discreto [4], de una forma diferente y en ese momento meramente teórico y, en consecuencia, útil para resolver el problema del logaritmo elíptico en campos finitos (ECDLP). Con algoritmos

clásicos estos problemas tienen una complejidad de tiempos exponencial para ser resueltos. Shor en cambio, con este artículo demostró que, con una computadora cuántica (teórica en ese momento), la complejidad de tiempo se convertiría en polinomial (N^3). Para ECDLP, la idea consiste en que el cálculo del algoritmo se base en encontrar periodos de la función matemática de la curva. Para esto, se deben aplicar transformadas de Fourier cuánticas (QFTs), que superponen e interfieren estados cuánticos para detectar patrones más rápido que lo que lo hacen los algoritmos clásicos.[4]

Contrario a la aproximación inicial que sugería una ausencia de implementaciones prácticas, la literatura reciente ha avanzado significativamente en estimaciones precisas y simulaciones de recursos. Roetteler et al. (2017 en Microsoft) establecieron estimaciones concretas para el algoritmo de Shor aplicado a curvas elípticas, determinando que para calcular logaritmos discretos en una curva sobre un cuerpo finito de n bits se requieren aproximadamente $9n + 2\lceil \log_2(n) \rceil + 10$ qubits lógicos y una profundidad de circuito dominada por $448n^3 \log_2(n)$ compuertas Toffoli. [20] Sin embargo, no corrieron el algoritmo en una QPU. Utilizaron un simulador de software llamado *LIQUI* ejecutado en computadoras clásicas o servidores de alto rendimiento, y así obtuvieron de resultado cuantos qubits se necesitarían para ejecutar un algoritmo y simularon la lógica para contar compuertas.

Asimismo, trabajos posteriores como los de Häner, Roetteler y Svore (2020) han optimizado estas arquitecturas logrando implementaciones de aritmética modular más eficientes que reducen la huella de qubits y la profundidad del circuito. [22] En este caso se optimizaron circuitos para simulaciones clásicas. Lograron simular más de 60 qubits en super-computadoras o clusters de CPUs/GPUs utilizando métodos matemáticos para comprimir la memoria.

Por su parte, investigadores como Gidney y Ekerå han refinado estos modelos teóricos incorporando costos realistas de corrección de errores, demostrando la viabilidad teórica de estos ataques bajo suposiciones físicas plausibles. [21] Pero tampoco utilizaron ninguna maquina cuántica real que implemente la funcionalidad que describen, en la que existe una corrección de errores completa. En su proyección arquitectónica mencionan que son necesarios “20 millones de qubits físicos” para ejecutar el algoritmo de Shor, tecnología que no existía a ese momento. [21]

Ventajas esenciales que presenta la criptografía de curva elíptica ECC, y por lo que hoy en día se sigue utilizando, las claves son mucho más cortas que la de los demás algoritmos, y presentan seguridad mayor y eficiente. [3] Se logró contrarrestar ataques existentes hasta la época, como el ataque de tiempo sub-exponencial del cálculo de índices contra problemas de logaritmo discreto estándar. Este menor riesgo permite que el tamaño de clave sea menor, y por ende tenga menor complejidad para generar la clave, menor espacio para almacenarla y menor ancho de banda para transmitirla. [11] Este criterio es fundamental hoy en día para su uso en dispositivos con recursos limitados, como los teléfonos móviles, o cualquier dispositivo IoT. [11]

Por el lado clásico, un ataque a ECC implica probar cada posible clave que pueda resolver el problema del logaritmo discreto elíptico. La cantidad de operaciones que implica esto en curvas estándar publicadas es inviable de resolver. Aun utilizando miles de computadoras, cada una procesando en paralelo, se tardarían millones de años en hallar la clave.[3]

5 Marco teórico

5.1 Criptografía

La criptografía es una ciencia, consiste en tomar un mensaje y hacerlo ilegible. También se la considera como el arte de esconder mensajes. Pero, no solo se trata de que esté protegido un mensaje, sino la forma en la que se implementa un algoritmo para protegerlo. [17]

5.1.1 Criptografía Asimétrica

Los algoritmos de clave asimétrica están diseñados de tal manera que un texto cifrado con la clave pública de un usuario solo puede descifrarse utilizando la clave privada del mismo usuario. A la inversa, si un texto se cifra con la clave privada de un usuario, solo puede descifrarse con la clave pública del mismo usuario. Por ello, estos algoritmos se denominan algoritmos de clave asimétrica. Las claves pública y privada correspondientes son diferentes, pero están matemáticamente asociadas. No es posible derivar una de ellas a partir de la otra. [17]

Basados en las características y estructuras de los algoritmos de clave asimétrica, estos algoritmos se emplean en operaciones de cifrado para mantener la confidencialidad o integridad de los datos. La confidencialidad se refiere a impedir que personas no autorizadas lean los datos, mientras que la integridad se refiere a impedir que personas no autorizadas los modifiquen. Durante la operación de cifrado, el remitente cifra los datos (denominados texto plano) utilizando la clave pública del destinatario (disponible para todos) y envía los datos cifrados (denominados texto cifrado). Cuando el destinatario recibe los datos cifrados, puede descifrarlos utilizando su clave privada, accesible solo para este usuario. Por lo tanto,

los datos se mantienen confidenciales durante su transmisión, incluso a través de canales no seguros. [17]

La mayoría de los algoritmos criptográficos de clave asimétrica se diseñan basándose en problemas matemáticos complejos, como la factorización de números primos y los logaritmos discretos, en los que el tiempo y el coste computacional para resolverlos se basan en funciones exponenciales (es decir, con un aumento lineal del tamaño de la clave, el tiempo y el coste computacional aumentan exponencialmente). Debido a las complejas operaciones matemáticas que realizan estos algoritmos, la mayoría requiere una cantidad significativa de tiempo y recursos computacionales para el cifrado y el descifrado.

5.2 Criptografía de curva elíptica

La criptografía de curva elíptica (ECC) es una técnica de cifrado asimétrico o de clave pública basada en la teoría de la curva elíptica, que se puede utilizar para crear claves criptográficas rápidas, pequeñas y eficientes. La idea fue postulada en 1987 por Neal Koblitz y Victor Miller. La propuesta consistía en utilizar curvas elípticas sobre campos finitos para implementar criptosistemas. [11]

Los métodos basados en curvas elípticas tienen su fortaleza en que por ahora no existen técnicas computacionales eficientes que permitan resolver el logaritmo elíptico. Pero como todos los métodos de criptografía asimétrica, la amenaza persistente de la computación cuántica lo pone en jaque. [3]

El método utiliza una curva elíptica de la forma:

$$y^2 = x^3 + ax + b$$

Ecuación 1 – ecuación matemática de la curva elíptica [3]

Cuando un conjunto de puntos satisface la *Ecuación 1*, crean una curva elíptica. Los valores de a y b son constantes, mientras que x e y son variables. Además, la curva permanece igual porque cualquier punto en ella se puede reflejar en el eje X e intersectarse con una línea no vertical en tres lugares o menos. [3]

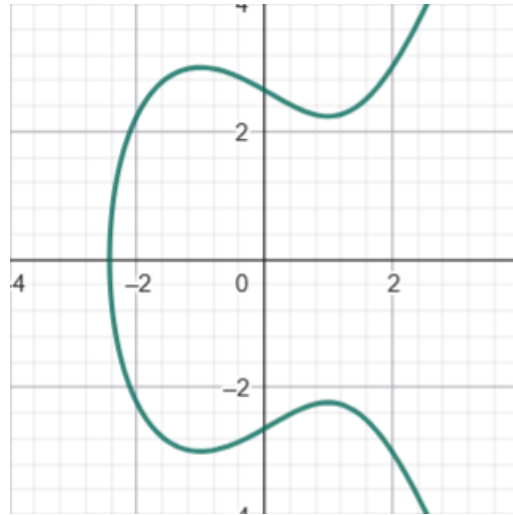


Imagen 2 – Grafico de curva elíptica de ecuación $y^2 = x^3 - 2x + 7$ – Fuente: elaboración propia utilizando GeoGebra [12]

La *Imagen 2* muestra el grafico de una curva elíptica definida en los números reales, esto quiere decir que su imagen se extiende hacia el infinito. El problema que conlleva dicha ecuación es que ninguna computadora clásica podría procesar y/o almacenar la totalidad de los puntos de la curva hasta el infinito y menos transmitirlos. [13]

El primer paso en ECC es elegir una de las curvas del dominio, para ello todas las partes que participan en el protocolo de cifrado y descifrado del mensaje, deben establecer un consenso para elegir la curva. La realidad es que este proceso de ponerse de acuerdo sobre la curva está limitado a un conjunto concreto de curvas, ya que son curvas probadas y que se consideran seguras. La curva entonces, en nuestro criptosistema, es un parámetro público y conocido.[11]

Por esto motivo NIST en su artículo NIST SP 800-186, recomienda algunas ecuaciones de curvas elípticas. En ese documento se proporciona una lista única y autorizada de parámetros de curvas seguras para ser utilizadas por parte del gobierno federal de Estados Unidos. Cada una de ellas fue examinada por NIST para asegurar que cumplen con los requisitos conocidos de seguridad de la información. [14]

Se denominan parámetros del dominio de la curva, a un conjunto de datos que se utilizan para definir una curva elíptica.

Estos parámetros se deben elegir con especial cuidado, ya que no todas las curvas poseen la misma fortaleza, por lo que es recomendable utilizar curvas con parámetros de dominios que ya estén validados. [11]

La criptografía de curva elíptica no trabaja con todos los números reales, sino que define a la curva en un grupo finito de tamaño fijo. Esto le brinda la seguridad requerida porque si no hay un límite en la composición de la curva, teniendo los puntos de inicio y fin es claro cuál es el valor de k . Sin embargo, definirlo en un campo finito hace que el k sea impredecible, debido a la cantidad de saltos que puede haber dado la curva sobre si misma hasta llegar al punto final. [13]

Entonces, la ecuación de la curva elíptica debe respetar la forma:

$$y^2 = x^3 + ax + b \bmod(p)$$

Ecuación 2 – Curva elíptica definida en un cuerpo finito [3]

Al estar definida en un campo finito según la *Ecuación 2*, la relación entre los componentes criptográficos y los elementos de la curva está representada en la siguiente ecuación, que es la base sobre la que se construye la criptografía de curva elíptica:

$$P = k * G$$

Ecuación 3 – Problema del logaritmo discreto de curvas elípticas (ECDLP) [3]

Donde en la *Ecuación 3* P y G son puntos de la curva, y k es un entero que define el número de iteraciones que se van a realizar sobre el punto G para obtener el punto P . Esta es la base de la seguridad que comprende la criptografía por curvas elípticas. [3] Como mencionamos anteriormente, definir en un cuerpo finito hace que, si bien son públicos los puntos P y G , hallar el k que relaciona ambos no es un proceso lineal, sino que requiere de hallar un patrón de recorrido por iteraciones sobre todos los puntos de la curva.

Las curvas parten de un punto base o generador denominado G , cada curva tiene un conjunto de estos puntos, los cuales poseen el máximo orden posible en esa curva, o n . El orden n de la curva es la cantidad de puntos iterables desde G hasta llegar al punto de origen o infinito definido en esa curva ϕ . El punto G que se tome como candidato debe ser el mismo para todos los usuarios que utilicen la curva y, por tanto, lo podemos considerar como un dato público. El emisor del mensaje elige un número k tal que $k < n$, es su clave privada, este número no puede ser revelado a nadie, ya que es el único dato que debe permanecer en secreto. El punto P es la clave pública y se genera a partir de la clave privada k multiplicada escalarmente al punto base G . [3]

Hasta ahora, entonces, los datos públicos comprenden tanto a la ecuación de la curva elíptica, que pertenece a un conjunto de curvas validadas y certificadas como “seguras” en el dominio de curvas elípticas; el punto base G , que es conocido y depende de cada curva elíptica correspondiente al dominio; y, por último, el punto resultante del problema del logaritmo discreto de curvas elípticas P . La clave privada del algoritmo de cifrado por curvas elípticas es el valor de k . [11]

En curvas elípticas se dispone de dos operaciones para calcular, una es la suma de puntos definidos en la curva, y la otra es la multiplicación escalar de un número o valor constante por un punto de la curva. [3]

La suma de puntos definidos en la curva elíptica forma un grupo abeliano porque cumple con las propiedades: [3]

- Conmutativa: $P + Q = Q + P$.
- Elemento inverso: $P + Q = 0$ si $Q = -P$.
- Elemento neutro: $P + 0 = P$
- Asociativa: $(P_1 + P_2) + P_3 = P_1 + (P_2 + P_3)$

La multiplicación escalar de un número por un punto de la curva cumple además con las siguientes propiedades: [3]

- $N = 2 * P = P + P$
- $N_4 = 4 * P = P + 3 * P = P + P + 2 * P = P + P + P + P$

Entonces, existe una función que define la curva. Se trabaja con un cuerpo finito de elementos que pertenecen a la curva. Los elementos del cuerpo se generan desde un punto de la curva, que se denomina generador. Todos los puntos para tener en cuenta están generados por el mismo punto y se generan en base a iteraciones sobre ese punto generador.[3]

Con este método podemos generar el cuerpo de elementos pertenecientes a la curva y que han sido generados con el punto generador G . Mediante una serie es posible calcular todos los puntos de la curva en base al generador G .

La serie que define esto es la siguiente (ejemplo ilustrativo): [3]

- $P_0 = G$
- $P_1 = G + P_0 = 2 * G$
- $P_2 = G + P_1 = 3 * G$
- $P_3 = G + P_2 = 4 * G$
- ...
- $P_{n-1} = G + P_{n-2} = n * G$

Definidas las operaciones, podemos concluir en que cualquier punto del cuerpo se puede calcular multiplicando al punto generador G por un número entero de la forma de la *Ecuación 3* mientras que el número se encuentre en el rango de 0 y el orden de la curva. [3]

La complejidad radica en que, si bien es posible realizar la operación de la *Ecuación 3*, que es una multiplicación escalar, la operación inversa para obtener el valor de k no se puede realizar. No es posible realizar la división $k = P/G$. [3]

Si bien obtener k es muy costoso, iterar k veces el punto G hasta obtener el punto P también sería inviable porque para una clave k de 354 bits se tardarían miles de millones de años. Para calcular P , se utiliza un método que requiere como máximo 256 operaciones. [3]

Suponiendo que $k = 356$, la serie se calcula utilizando el elemento anterior (ejemplo ilustrativo): [3]

- $2^0 * G = G$
- $2^1 * G = 2^0 * G + 2^0 * G = G + G$
- $2^2 * G = 2^1 * G + 2^1 * G = 2 * G + 2 * G$

- $2^3 * G = 2^2 * G + 2^2 * G = 4 * G + 4 * G$
- $2^4 * G = 2^3 * G + 2^3 * G = 8 * G + 8 * G$
- $2^5 * G = 2^4 * G + 2^4 * G = 16 * G + 16 * G$
- ...
- $2^{255} * G = 2^{254} * G + 2^{254} * G$

k se pasa a binario y se calcula: [3]

- 0101100100
- $2^8 + 2^6 + 2^5 + 2^2 = 356$

Con todo esto, se construye la suma de los términos necesarios: [3]

- $P = 2^8 * G + 2^6 * G + 2^5 * G + 2^2 * G = 356 * G$

De esta manera, calculamos P de manera sencilla y rápida, sin la necesidad de iterar k veces sobre G . De aquí radica la potencia de las curvas elípticas, con ciertas operaciones se puede obtener resultados con muy poco costo de tiempo, pero realizar las operaciones inversas tienen un costo demasiado elevado. [3]

5.3 Algoritmo de Shor

Peter Shor, como fue mencionado en la sección 4, publica un algoritmo que se puede implementar en un circuito cuántico capaz de resolver el problema del logaritmo discreto y la factorización de números grandes. [4]

La idea de Shor fue crucial ya que parte de un problema complejo (que sustenta la criptografía asimétrica o de clave pública), para reducirlo en un único problema de búsqueda de periodos (period-finding), campo en el cual se destaca la computación cuántica. [4] Entonces, el problema del logaritmo discreto en curvas

elípticas ECDLP, que fue detallado en la sección 5.2 enfrenta al problema cuántico de la búsqueda del periodo.

En resumen, dada una cierta función $f(x)$, en forma de caja negra, para encontrar su periodo r , tal que $f(x) = f(x + r)$, se puede utilizar una computadora cuántica, donde el periodo de la función dependa de la clave secreta k . [4]

La ecuación de la función periódica tiene la forma:

$$f(a, b) = a * G + b * P$$

Ecuación 4 – Función Oráculo [4]

En la *Ecuación 4* definimos a G como el punto generador de la curva, y P es la clave pública generada por la *Ecuación 3*. La función toma dos puntos de entrada (a y b). A esta función se la denomina “Oráculo”. [4]

El momento en que empieza a actuar la computación cuántica, y su ventaja, es cuando se requiere probar todos los (a, b) disponibles uno por uno. La computadora cuántica puede preparar dos registros de qubits en una superposición de todos los pares (a, b) posibles de la curva, todo al mismo tiempo en un solo estado coherente. [4] La computadora cuántica entonces, ejecuta la función Oráculo en un estado de superposición, calculando todos los resultados de $f(a, b)$ posibles simultáneamente. [4]

Al finalizar ese proceso, y como pilar fundamental de la cuántica, se ejecuta la transformada cuántica de Fourier (QFT) (ver sección 5.4), la cual recibe los resultados y estados resultantes que entrega el cálculo de la función Oráculo en superposición. Algunos de esos resultados se repiten, lo cual quiere decir que son periódicos. La función de la QFT es encontrar patrones y la periodicidad de los resultados. Su metodología se basa en destruir aquellos caminos que no son parte del

patrón a descubrir, y amplificar de manera constructiva las respuestas que sí lo son. [4]

Como paso final, se debe medir el estado cuántico resultado de la QFT. La superposición colapsa con una probabilidad relacionada a un valor que sobresale con respecto a las probabilidades de los demás valores. Este valor está directamente relacionado con el periodo de la función. [4]

Este periodo resultado representa una relación dada por un par de números (a, b) tal que $f(a, b)$ colisiona, es decir, que hacen que la función llegue al punto de identidad de la curva, o punto en el infinito Φ . Este resultado se representa con la siguiente ecuación: [4]

$$f(a, b) = a * G + b * P = \Phi$$

Ecuación 5 – Función Oráculo para hallar el patrón periódico de la curva [4]

A constitución, detallaremos los pasos derivativos que concluirán en la *Ecuación 6*. Sustituyendo P con la *Ecuación 4*: [4]

$$f(a, b) = a * G + b * (k * G) = \Phi$$

Agrupando los términos y utilizando propiedades de la aritmética de curvas elípticas:

$$(a + b * k) * G = \Phi$$

Esta ecuación solo es cierta si y solo si se cumple que $(a + b * k)$ es múltiplo del orden de la curva r . Esto quiere decir, que debe condicionarse a:

$$a + b * k \equiv 0 \pmod{r}$$

Ya en este punto, tras la medición, los valores de a y b son conocidos; el orden r de la curva con el generador G son públicos, y nos falta despejar la k utilizando métodos clásicos en un post procesado de la computadora cuántica. Debemos resolver:

$$b * k \equiv -a \pmod{r}$$

$$k \equiv (-a) * (b^{-1}) \pmod{r}$$

Ecuación 6 – Resultado del algoritmo de Shor en la computadora cuántica [4]

Llegado hasta aquí, la *Ecuación 6* es la solución del problema del logaritmo discreto en curvas elípticas ECDLP utilizando computación cuántica. [4]

5.3.1 Transformada Cuántica de Fourier (QFT)

El algoritmo de Shor funciona gracias a una subrutina llamada estimación de fase. El objetivo de esta estimación es hallar un patrón en las operaciones matemáticas usadas para buscar la clave. Al realizar estas operaciones repetitivas, los resultados, lejos de ser aleatorios, comienzan a repetirse en forma de patrón u onda. La idea es encontrar cada cuánto se repite ese patrón (periodo) y con ese dato hallar la clave. Sin embargo, este trabajo implica operar con un universo gigante de números; hacerlo de manera clásica es inviable, ya que se debería medir la onda punto por punto para reconstruirla y detectar cuándo se repite. [18]

En este contexto, la computación cuántica no analiza los números individualmente, sino la frecuencia con la que se repiten. De esto se encarga la Transformada Cuántica de Fourier (QFT), un algoritmo de cambio de base cuya función es tomar toda la información almacenada simultáneamente (gracias a la superposición) y manipular sus probabilidades. La QFT aplica una transformación global con dos efectos: la interferencia constructiva, que identifica los patrones que

se repiten y hace que las probabilidades de los datos correctos se sumen y amplifiquen; y la interferencia destructiva, que hace que las probabilidades del ruido (datos que no siguen el patrón) se resten entre sí y se anulen. [18]

En resumen, la QFT reordena matemáticamente las ondas de probabilidad para que la respuesta correcta resalte y las incorrectas desaparezcan. Antes de aplicar la QFT, la información está codificada como valores individuales; luego de su aplicación, los datos se codifican como frecuencias. Al medir esta frecuencia de repetición es donde se recupera el valor de la clave. [18]

6 Metodología de trabajo

Comenzando con el desarrollo del presente trabajo, se detalla a continuación el paso a paso necesario con el objetivo de obtener resultados medibles y que permitan la comparación. Para comparar los resultados del experimento, es preciso elegir una serie de métricas que sean determinantes para el análisis y la decisión de las conclusiones.

Las métricas que se evalúan comparativamente en tanto al desempeño de la ejecución clásica versus cuántica son:

- 1) Tiempo de ejecución. Medir el tiempo de ejecución del circuito cuántico en la QPU provista por IBM. Del lado clásico, calcular el tiempo de atacar por fuerza bruta a todo el conjunto de claves posibles generados por la curva elíptica (equivalentemente al algoritmo de Shor).
- 2) Complejidad computacional. Clasificar teórica y prácticamente la dificultad de resolver el problema del logaritmo discreto en curvas elípticas de las formas clásica y cuántica.

Para la misma ecuación (ECC) se compara los resultados obtenidos luego de ejecutar un ataque clásico versus los resultados de un ataque cuántico utilizando la función de Shor, los cuales sirven para comparar ambas formas de romper una clave ECC. Además, hay que definir los parámetros de curva que se van a utilizar.

Cabe destacar que los resultados del experimento se obtienen utilizando una computadora que tiene las siguientes características y especificaciones:

- Procesador: Intel (R) Core (TM) i7-1065G7 CPU @ 1.30GHz (1.50 GHz).
- Placa gráfica: NVIDIA GeForce MX230.

- Memoria RAM: 32,0 GB (31,8 GB usable).
- Memoria: Discos 932GB HDD y 447GB SSD.
- OS: Windows 11.
- Entorno de ejecución: Jupyter Notebook.

Por el otro lado, para el experimento en la computación cuántica se agregan las siguientes métricas:

- Cantidad de qubits requeridos (representa el *width* o amplitud del circuito).
- Costos económicos por utilizar los servicios de IBM.

Es fundamental destacar que, a diferencia de los trabajos de Roetteler et al. [20] o Gidney y Ekerå [21], que fundamentan sus resultados en simulaciones clásicas de alto rendimiento o estimaciones teóricas proyectadas para un futuro hardware tolerante a fallas (FTQC), esta investigación se distingue por su enfoque empírico sobre hardware cuántico real de escala intermedia (NISQ). Mientras que la literatura existente tiende a idealizar el comportamiento de los qubits lógicos mediante modelos matemáticos, este trabajo enfrenta las limitaciones físicas inherentes al estado actual de la tecnología, como el ruido de compuerta y la decoherencia, utilizando la infraestructura de acceso abierto de IBM Quantum. Por consiguiente, este enfoque cubre un vacío en el estado del arte actual, contrastando las proyecciones teóricas idealizadas con la viabilidad práctica de ejecución en los procesadores físicos disponibles hoy en día.

El desarrollo del código para la ejecución de las pruebas para el presente trabajo se realizó en el IDE de Visual Studio Code, mediante la creación de archivos de tipo Jupyter ipynb. El potencial brindado por esta herramienta y todas las librerías

de Python, haciendo énfasis en la compatibilidad con Qiskit, fueron las principales razones de la elección del lenguaje de programación. [7]

Para comenzar a medir las métricas y desarrollar el código, se toma de ejemplo la ecuación de curva elíptica “secp256k1” publicada como estándar (la que se usa en Bitcoin y Ethereum). [15] La misma tiene ecuación $y^2 = x^3 + 7$, donde $a = 0$ y $b = 7$. El valor del módulo primo p , que define el cuerpo finito de la curva, se itera progresivamente para aumentar la cantidad de puntos disponibles que tiene la curva y así poder aumentar el tamaño de la clave en bits.

La elección de los parámetros de la curva tiene su origen en su estandarización por motivos de ser lo que se denomina una curva Koblitz ($a = 0$), con eficiencia en la rapidez de implementación y su transparencia en la elección de parámetros ($b = 7$). [15] El artículo de SECG además resalta que la velocidad de implementación de la curva es casi dos veces más rápida en la multiplicación escalar de las curvas definidas por NIST. [15]

El módulo p que define que la matemática de la curva sea un cuerpo finito es fundamental en ECC. La elección de este requiere que sea un número primo, a medida que va iterando sobre los parámetros de la curva, el valor de p va a aumentar, siempre respetando que sea un número primo. [11]

6.1 CrypTool

CrypTool es un software de código abierto diseñado para analizar la criptografía y el criptoanálisis. Funciona como un laboratorio de pruebas de herramientas de criptografía. Permite utilizar algoritmos, romperlos, visualizar conceptos, etc. Al permitir visualizar a baja escala la distribución de puntos de curvas

elípticas, y el resultado de la suma de puntos y multiplicación escalar (*Ecuación 3*), toma importancia al principio como referencia para desarrollar los programas. [16]

Para módulos p menores a 100 funciona el software CrypTool, que, junto con Java, brinda la información de las curvas elípticas, todos los puntos generados, y las operaciones posibles que se pueden realizar a partir de tomar cualquier punto como generador G . De esta manera se obtienen todos los puntos P posibles de la curva mediante la multiplicación escalar y la suma de puntos. Con esto, se obtiene de forma gráfica la visualización de una curva elíptica definida en un campo finito. La herramienta entonces permite visualizar la gráfica de curvas con baja latencia para parámetros pequeños. [16]

En la *Imagen 3* se brinda la gráfica de la curva elíptica definida en un grupo finito o discreto dado por la ecuación $y^2 = x^3 + 7 \bmod(97)$. La misma genera un orden n de 79 puntos sobre los cuales podemos iterar, es decir que es posible tomar cualquier k que cumpla con $0 < k < n$. [11]

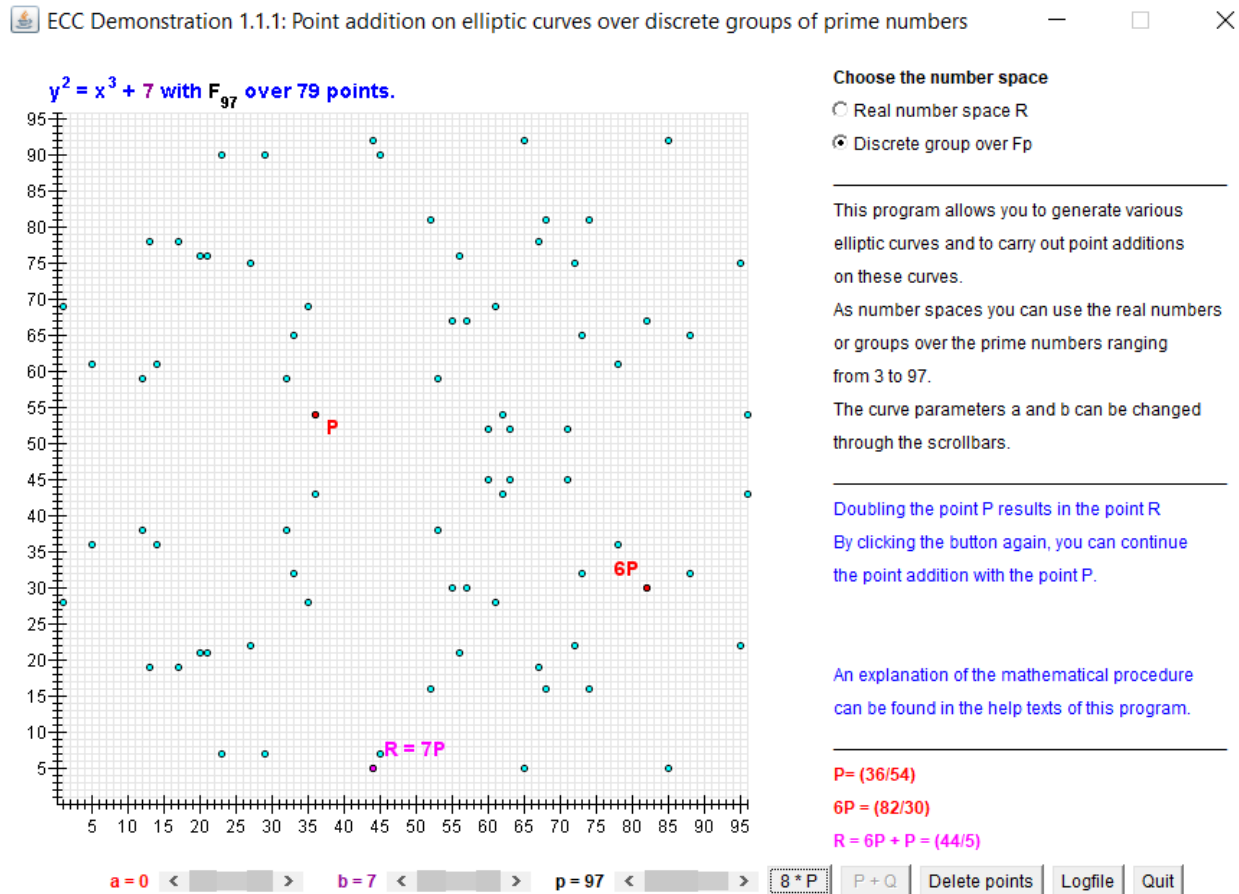


Imagen 3 – Curva elíptica definida en un cuerpo finito – Fuente: elaboración propia utilizando CrypTool [16]

En el ejemplo se logra ver que P (en este caso es el generador G) = $(36, 54)$ y $6P$ (en este caso es el punto P de la clave pública) = $(82, 30)$.

Además, la misma herramienta brinda la gráfica de la curva elíptica en el espacio de los números reales, como se puede apreciar en la Imagen 4, donde no se define un grupo finito de la curva dependiendo del $\text{mod}(p)$.

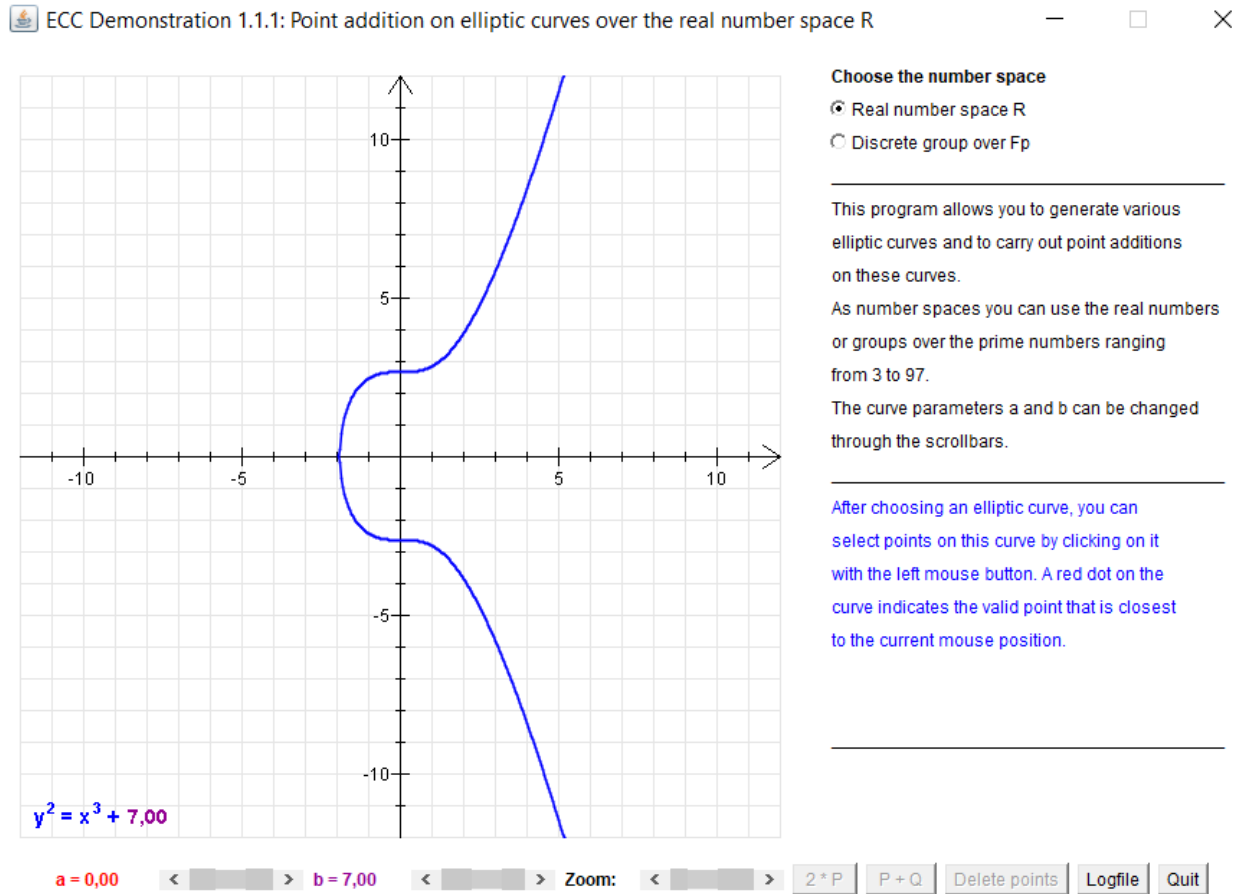


Imagen 4 – Curva elíptica definida en los reales – Fuente: elaboración propia utilizando CrypTool [16]

Si bien la herramienta ofrece una interfaz muy útil para comenzar a trabajar con las curvas elípticas, para módulos $p > 97$, es necesario desarrollar un programa de Python capaz de, con solo la ecuación de la curva y el módulo como input, permita devolver una lista de puntos y el orden n máximo de esos puntos de la curva. El mismo se puede ver en el *Anexo 3* (ver *sección 11.3.1*).

6.2 Qiskit

Quantum Information Science Kit es un desarrollo de software de código abierto basado en Python, creado y mantenido por IBM. El objetivo principal de

Qiskit es permitir a los usuarios crear, simular y ejecutar programas definidos en circuitos capaces de resolverse en computadoras cuánticas. [7]

IBM, a través de su plataforma, Qiskit - IBM Quantum Platform (de ahora en más denominada Qiskit), permite a los usuarios realizar pruebas donde los primeros 10 minutos mensuales de ejecución en *backend* real QPU son gratuitos. Esto permite experimentar con computadoras cuánticas reales, adicionalmente Qiskit permite interactuar con un *backend* físico y ejecutar circuitos cuánticos desarrollados con programación en Python, para medir resultados. [7]

Luego de utilizar esos 10 minutos mensuales que comprende el periodo de facturación de la plataforma de Qiskit, es posible contratar el servicio con el costo de 1,60 US\$ por segundo extra de uso del *backend* físico. Este costo lo aplica como métrica al momento de conseguir los resultados con la computación cuántica. [8]

La interfaz de IBM Quantum Platform sirve para la creación de instancias para utilizar *backends* físicos mediante Qiskit. Además, permite realizar un seguimiento del uso mensual de las instancias. La forma de conectar el ejecutable con estas instancias es a partir de una API Key única por usuario, que se carga en el entorno de este mediante la librería *Qiskit Runtime Service*¹. [8]

Algunos términos para tener en cuenta son:

- *Backend*: es el destino de ejecución del código, es decir, la computadora que lo ejecutará. Puede ser un simulador o un sistema cuántico real. El simulador finge ser una computadora cuántica, no tiene ruido y es ideal para depurar el circuito antes de ejecutarlo en un sistema real. Las computadoras reales tienen

¹ Ver Anexo 1: Qiskit Runtime Service

ruido, colas de espera para ser utilizadas, y son las que realmente se utilizan para medir la capacidad de la computación cuántica. [7]

- *Transpilation* (transpilación): es el proceso de compilación, el cual traduce el circuito desarrollado con su comportamiento ideal en un circuito optimizado para ejecutar en un *backend* físico específico. Un sistema cuántico real tiene limitaciones, como la topología (no todos los qubits están conectados entre sí), o las denominadas compuertas nativas, dependiendo del *backend*, podrá ejecutar un conjunto limitado de compuertas. [7]
- *Preset Pass Manager* (gestor de pases predefinidos): es una herramienta que maneja la etapa de transpilación, de manera tal que mapea los qubits, añade compuertas en caso de ser necesario para la comunicación entre los qubits, descompone compuertas complejas en nativas y optimiza la redundancia del circuito para achicarlo y reducir el posible ruido que pueda sufrir. [7]
- *Job* (trabajo): es un registro único que guarda la solicitud de ejecución en el *backend*. Permite ver el estado de la solicitud, conocer la posición en la cola de ejecución, cancelar el trabajo y recuperar los resultados. [7]
- *Sampler* (muestreador): es una interfaz de alto nivel que brinda la distribución de probabilidades de salida del circuito. Gestiona los *shots* o ejecuciones, el trabajo y puede mitigar la propagación de errores por resultados ruidosos. [7]

La plataforma, como interfaz, también brinda un “*composer*”, que sirve para desarrollar y previsualizar circuitos en tiempo real con todas las compuertas disponibles. En la *Imagen 5* se ve a grandes rasgos el circuito cuántico que aplica el algoritmo de Shor utilizando solo 4 qubits (2 registros con 2 qubits cada uno).

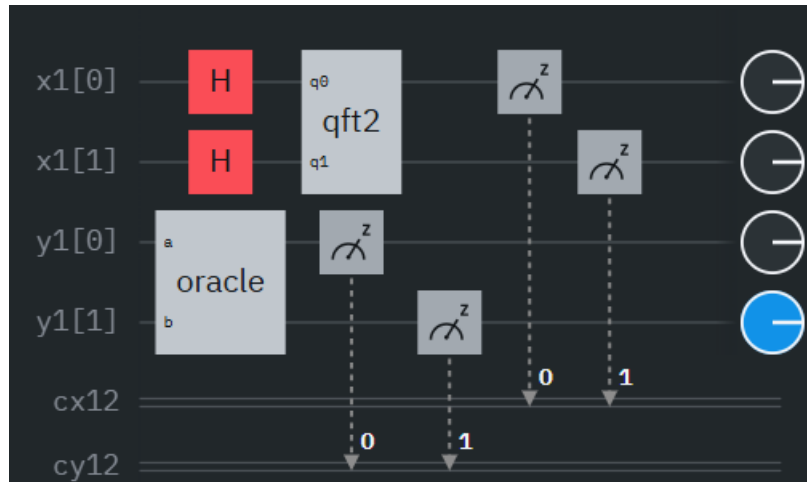


Imagen 5 – Circuito modelo del algoritmo de Shor – Fuente: elaboración propia utilizando IBM Quantum Platform [8]

La *Imagen 5* es un ejemplo a escala de juguete que implementa las tres etapas clave del algoritmo de Shor, la superposición, la llamada a la función Oráculo, y la transformada de Fourier cuántica QFT. [11] El mismo se desarrolla en el lenguaje de programación OPENQASM 2.0² y la misma herramienta traduce este código en lenguaje Python para ejecutar con Qiskit. [8]

Se destaca en la *Imagen 5* la presencia de:

- Registros de conteo ($x1[0]$ y $x1[1]$): es el primer registro cuántico compuesto por 2 qubits. Almacena la entrada x de la función a analizar. Gracias a las compuertas Hadamard, almacenará todas las entradas posibles al mismo tiempo en superposición (1, 2, 3, 4). En el final de la ejecución, luego de la QFT, este es el registro que se mide para encontrar el periodo. [8]

² Ver Anexo 2: OPENQASM 2.0

- Registros de trabajo ($y1[0]$ y $y1[1]$): es el segundo registro cuántico. Almacena el resultado del cálculo de la función Oráculo. [8]
- Compuertas *Hadamard*: es una compuerta fundamental de la computación cuántica, que transforma los qubits bien definidos (en estado 0 o 1) en una superposición equitativa de estados con misma probabilidad. [8]
- Compuerta *Oracle*: es la caja negra que ejecuta la función Oráculo. La computación cuántica ejecuta esta compuerta en todos los estados de superposición de $x1$ simultáneamente. Oracle enreda el registro $x1$ con $y1$, y resulta en el periodo que queda impreso como patrón de fase en $x1$. [8]
- Compuerta QFT2: ejecuta la transformada inversa de Fourier cuántica para 2 qubits (en Shor se usa la inversa, que cambia el signo de las rotaciones). [8]
- Compuertas de medición: en la *Imagen 5* se ven con el símbolo “Z”. Miden los qubits, colapsando su superposición a un valor clásico binario (0 o 1). La medición crucial es la de $x1$, que va a los bits $cx12$. [8]
- Registros clásicos ($cx12$ y $cy12$): son los registros clásicos (bits) que almacenan los resultados de la medición. $cx12$ contiene el resultado de medir $x1$ (ejemplo de salida: “01”). Con este resultado se puede deducir el periodo r con un post procesamiento clásico, y con el periodo calcular la clave k . [8]

En la *Imagen 6* se ve cómo se logra la distribución de probabilidades.

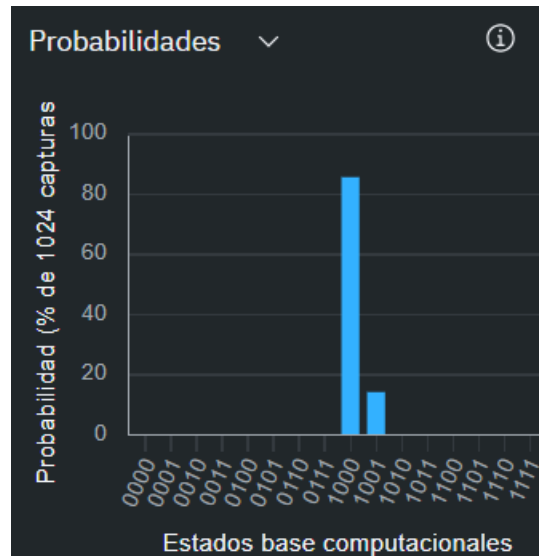


Imagen 6 – Distribución de probabilidades con 1024 shots – Fuente: elaboración propia utilizando IBM Quantum Platform [8]

Como es posible apreciar, el registro “1000” (8 en binario) es el que tiene mayor probabilidad de ser el período r objetivo de búsqueda. [11]

Si bien estas herramientas brindan un contexto y una visual al diseño, no permiten escalar la funcionalidad en cantidad de qubits y tampoco admite procesamiento clásico para evaluar los resultados dentro de la misma plataforma. Por estas mismas razones es que el desarrollo se realiza mediante Qiskit con interfaz de uso de Visual Studio Code y Jupyter Notebooks.

Para visualizar los códigos ejecutables del desarrollo necesario para el procesamiento cuántico y el clásico, los mismos se encuentran en el *Anexo 3* (ver *sección 11.3*).

6.2.1 *Backend* utilizado: `ibm_marrakesh`

El backend utilizado para ejecutar el circuito cuántico mediante Qiskit y Qiskit Runtime Service es el `ibm_marrakesh`. Las características técnicas de este *backend* son las siguientes: [8]

- Qubits disponibles: 156. Este es el número de bits cuánticos que se utilizan para hacer los cálculos. [8]
- Region: Washington DC (us-east). [8]
- Versión de QPU: 1.0.12. [8]
- Tipo de procesador: Heron r2. Heron es la familia más nueva de IBM, diseñada específicamente para tener menor tasa de error que las generaciones anteriores. [8]
- *Clops*: 200 mil. Esta QPU mide las operaciones por segundo en la capa de circuitos teniendo en cuenta el hardware. Es una medida de velocidad de ejecución que mide cuantas capas enteras de un algoritmo puede ejecutar por segundo. [8]
- Puertas de base: `cz`, `id`, `rx`, `rz`, `rzz`, `sx` y `x`. Representan el alfabeto de operaciones básicas que entiende el procesador. Cualquier algoritmo debe ser *transpiled* (transpilado) o traducido solo a este tipo de instrucciones. [8]

Es importante recalcar la calidad y el ruido que ofrece este *backend* con respecto a los demás disponibles en la versión gratuita. En definitiva, es el que menor tasas de error ofrece, y mayor cantidad de qubits disponibles. [8] Las tasas de error miden qué tanto ha errado la implementación (errores de respuesta entre 0 y 1). El backend ofrece una tasa de error del 0,367%, lo que demuestra la calidad de las operaciones de entrelazamiento. [7]

6.3 Desarrollo del Circuito Cuántico

Si bien CrypTool fue fundamental para la comprensión visual y la validación teórica de los puntos de la curva en la etapa temprana, para la experimentación masiva y automatizada se migró a scripts de Python personalizados descritos a continuación.

El desarrollo del programa comienza con la definición del problema matemático que se busca resolver, en este caso el problema del logaritmo discreto en curvas elípticas. El objetivo es resolver $P = k * G$ en una curva elíptica definida sobre un cuerpo finito. La curva tiene la ecuación $y^2 \equiv x^3 + a * x + b \pmod{p}$. Las entradas o parámetros son un punto generador G de orden r y un punto público P . La incógnita o el valor para buscar es la clave privada k . A partir de ese conjunto de datos, el problema que resuelve el algoritmo de Shor es el de hallar el periodo de la función $f(s, t) = s * G + t * R$, donde dado que $R = k * G$, la función es periódica respecto a la relación lineal de s y t .

Definidos los datos y el objetivo, cabe destacar que, debido a las limitaciones actuales de profundidad de circuito y tiempos de coherencia en los procesadores IBM Quantum (con ciclos de código de superficie estimados en microsegundos y tasas de error de compuerta del 0.1%), la implementación completa de la aritmética modular cuántica resulta inviable por el ruido acumulado. Por esta razón, se optó metodológicamente por un enfoque híbrido utilizando un oráculo con tablas de verdad pre-calculadas. Esta decisión permite validar el funcionamiento del núcleo del algoritmo de Shor (búsqueda de periodo y QFT) aislando la complejidad aritmética que requeriría hardware tolerante a fallas para ejecutarse sin decoherencia.

Se comienza con el desarrollo con la definición de los parámetros, luego una pre-computación clásica, en donde se generan clásicamente todas las combinaciones lineales posibles de s y t para calcular los puntos resultantes $R = s * G + t * P$. A cada uno de esos puntos se les asigna un identificador entero id . Además, clásicamente se codifican las sumas de todos los puntos en una tabla pre-calculada en donde el oráculo activa el registro objetivo id en caso de que los registros de control sean s y t .

Adentrando las fases del circuito cuántico, se inicializan los registros s y t aplicando compuertas *Hadamard*, lo cual crea una superposición equiprobable de todos los pares enteros posibles. Se realiza el entrelazamiento del Oráculo, donde el estado del sistema se entrelaza con el registro id , creando una correlación entre los coeficientes de entrada y el punto resultante en la curva. Se aplica la QFT en interferencia constructiva a los registros s y t para transformar la información codificada en amplitudes de probabilidad en el dominio de la frecuencia o fases. Esto construye aquellos estados que satisfacen la relación de periodicidad impuesta por la estructura de grupo de la curva elíptica. Por último, se miden los registros s y t , obteniendo cadenas de bits clásicas que representan valores enteros u y v .

Para finalizar, en este enfoque híbrido, es necesario un post-procesamiento clásico, en donde se analiza el resultado de la computación cuántica. La medición entrega pares (u, v) con probabilidades asociadas, en donde solo un par (u, v) satisface la relación ortogonal al vector del periodo oculto. Matemáticamente, queda resolver para cada par (u, v) la ecuación $v * k + u \equiv 0 \pmod{r}$. Despejando k , se obtiene $k_{candidato} = (-u * v^{-1}) \pmod{r}$. Se puede validar el resultado probando que $k_{candidato} * G = P$.

El desarrollo del programa se puede visualizar en el Anexo 3 (ver *sección 11.3.2*).

7 Resultados Obtenidos

Para entender los resultados, primero se detalla lo que devuelve cada método de ataque a una clave ECC de manera individual, y luego se los compara para obtener una conclusión sobre todo el trabajo realizado. Por lo tanto, se muestra primero cuáles fueron los tiempos de ejecución de cada uno, y las métricas que sirven para analizar.

7.1 Resultados computación clásica

Se comienza visualizando las métricas del ataque por fuerza bruta de la computación clásica. Con este método se toma en cuenta que, en el peor de los casos, se deben probar la totalidad de las claves posibles para la curva con la ecuación especificada y el orden dado por el módulo que define el grupo finito.

7.1.1 Tiempos Requeridos

La primera métrica que se mide es el tiempo de ejecución. El desarrollo utilizado para medir estos tiempos se encuentra en el *Anexo 3* (ver *sección 11.3*).



Imagen 7 – Tiempos de ataque por fuerza bruta clásica vs. el tamaño de clave en bits – Fuente: elaboración propia a partir de los datos experimentales obtenidos

Como se puede observar en el gráfico de la *imagen 7*, la curva, conforme aumenta el tamaño en bits de la clave, tiene un comportamiento exponencial en el valor del tiempo requerido para el ataque por fuerza bruta.

Se plasma este gráfico en los resultados, los tiempos en tabla se ven:

Tamaño de la clave k en bits	Tiempo en segundos
6	0,00260s
7	0,00471s
8	0,00652s
9	0,01551s
10	0,03896s
11	0,05382s
12	0,10471s
13	0,26509s
14	0,51323s
15	1,13776s
16	2,83673s
17	7,45398s
18	16,72867s

19	37,48334s
20	70,48966s
21	152,94489s
22	333,10265s

Tabla 1 – Resultados del ataque por fuerza bruta clásico – Fuente: elaboración propia a partir de los datos experimentales obtenidos utilizando el código desarrollado en Python (ver sección 11.3)

Como se puede ver en los datos de la *Tabla 1*, se aprecia un crecimiento exponencial de tiempos con respecto al aumento iterativo del tamaño de la clave.

7.1.2 Complejidad Computacional

Aplicando una regresión exponencial a los resultados, la complejidad en términos de tiempo responde a una relación exponencial del tipo $2,25^N$ promedio entre todas las muestras recabadas con respecto a la cantidad de bits en tamaño de la clave.

Para entender el significado, N es el tamaño de la clave, y 2,25 es resultado del valor de la tasa de crecimiento promedio de los tiempos conforme aumenta el tamaño de clave.

Al calcular la razón entre un paso y el anterior $Tiempo_N/Tiempo_{N-1}$, donde el ruido afecta menos al sistema, el resultado en promedio es 2,25. Por ejemplo, entre 21 y 22 bits de clave, los tiempos son: $333,10s/152,94s \approx 2,17$.

La interpretación es que se multiplica aproximadamente por 2,25 el tiempo cada vez que se aumenta en 1 la cantidad de bits de la clave. En términos técnicos, al sumar 1 a la entrada $N + 1$, el costo se duplica con potencia 2,25.

Cabe destacar que, teóricamente, un ataque por fuerza bruta binaria debería duplicar su tiempo con cada bit adicional (tasa de crecimiento de 2). El factor

experimental obtenido de 2,25 se atribuye a la sobrecarga computacional (*overhead*) intrínseca del entorno de ejecución (Python y Jupyter) y a la ineficiencia en la gestión de memoria al operar con listas y grandes enteros, factores que añaden latencia adicional más allá de la complejidad matemática pura del algoritmo.

7.2 Resultados Computación Cuántica

En el caso del ataque a curvas elípticas mediante el algoritmo de Shor, se toma el tiempo de QPU necesario para ejecutar el circuito cuántico.

7.2.1 Tiempos Requeridos

El tiempo de ejecución de la QPU conforme aumenta el tamaño en bits de la clave resulta tener la siguiente visualización gráfica:

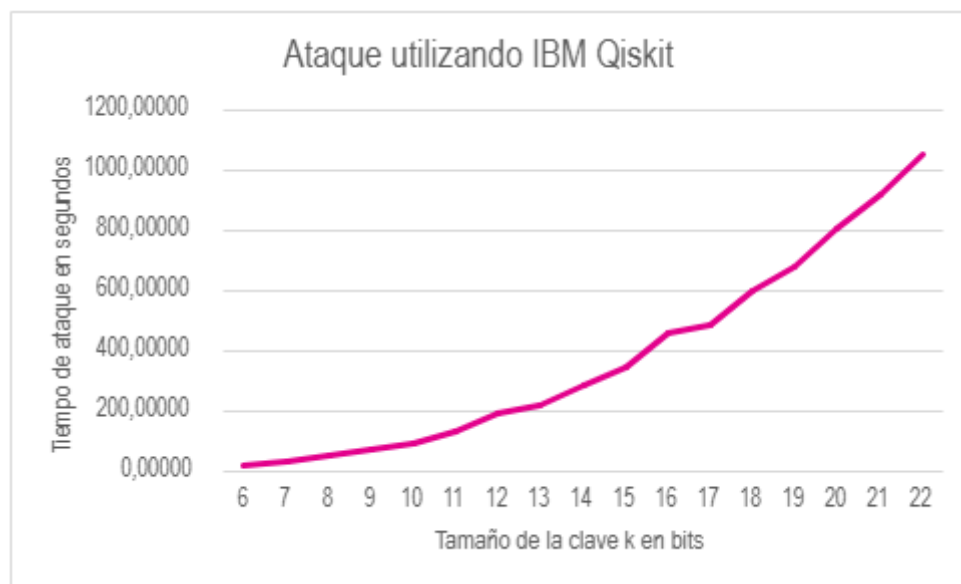


Imagen 8 – Tiempos de ataque con el algoritmo de Shor vs. el tamaño de clave en bits – Fuente: elaboración propia a partir de los datos experimentales obtenidos

La *Imagen 8* muestra una curva que responde a un crecimiento polinomial de los tiempos conforme aumenta el tamaño de la clave en bits. Pueden apreciarse

variaciones que no logran definir la perfección de la curva, esto es debido a ruidos en la comunicación o variaciones que pudieron demorar el normal funcionamiento de la ejecución.

Los resultados de los tiempos de esa curva se encuentran en la *Tabla 2*:

Tamaño de la clave k en bits	Tiempo en segundos
6	21,61106s
7	33,31757s
8	52,22622s
9	72,93733s
10	91,05121s
11	133,16817s
12	192,88850s
13	219,81252s
14	287,54053s
15	349,67285s
16	457,80977s
17	487,55161s
18	599,49868s
19	678,25127s
20	807,40971s
21	919,57429s
22	1057,34532s

Tabla 2 – Resultados del ataque con Shor – Fuente: elaboración propia a partir de los datos experimentales obtenidos utilizando el código desarrollado en Python (ver sección 11.3)

En comparación directa entre los valores de la *Tabla 1* y la *Tabla 2*, para todos los bits de clave analizados, la diferencia es a favor del ataque por fuerza bruta clásico. Para 22 bits, el ataque clásico demora 333,1s, mientras que, en el ataque con computadora cuántica, se demora 1057,3s.

7.2.2 Complejidad Computacional

Los resultados de la computación cuántica, junto con la *Imagen 8* de la gráfica muestran el comportamiento de una función polinomial de grado 3 del tiempo respecto de la cantidad de bits de la clave. Esto quiere decir que la complejidad y el aumento del tiempo conforme aumenta el tamaño de la clave en bits tiene la forma N^3 .

La evidencia de este resultado se ve en la tasa de crecimiento, en el caso de aumentar la clave de 21 a 22 bits, el tiempo solo aumenta un 15%, o a un factor de 1,15, lo cual es consistente con la relación $\left(\frac{22}{21}\right)^3 \approx 1,15$.

7.2.2.1 Cantidad de Qubits

También es métrica de análisis la cantidad de qubits necesarios para cada uno de los tamaños de clave del experimento. La curva de cantidad de qubits necesarios con respecto a la cantidad de bits de la clave corresponde a la del siguiente gráfico:



Imagen 9 - Curva definida por los qubits requeridos vs. el tamaño de clave en bits – Fuente: elaboración propia a partir de los datos experimentales obtenidos

La *Imagen 9* muestra una variación lineal entre los qubits necesarios para atacar a cada clave conforme aumenta su tamaño en bits. Los datos se visualizan en la siguiente tabla:

Tamaño de la clave k en bits	Qubits necesarios
6	42
7	49
8	56
9	63
10	69
11	76
12	83
13	90
14	97
15	104
16	111
17	118
18	125
19	132
20	138
21	145
22	152

Tabla 3 – Cantidad de qubits necesarios vs. tamaño de clave en bits – Fuente: elaboración propia a partir de los datos experimentales obtenidos utilizando el código desarrollado en Python (ver sección 11.3)

En el grafico de la *Imagen 9* y la *Tabla 3* es posible apreciar que la complejidad en este campo es polinomial también, en este caso de grado 1, respetando una forma N . Es decir que la complejidad espacial, traducida en uso de recursos de hardware cuántico, crece proporcionalmente al tamaño de la clave en bits.

7.3 Análisis y Analogía de resultados

En esta sección se comparan ambos resultados para poder visualizar de manera más fácil y clara las diferencias obtenidas entre ambos métodos de ataque con las métricas definidas:

7.3.1 Tiempos Requeridos

Se comienza con la primera métrica: **tiempos requeridos**, que es el factor clave de análisis del presente trabajo.

Partiendo en el rango comprendido entre tamaños de clave de criptografía de curva elíptica de 6 a 22 bits (experimento), se puede apreciar que el ataque clásico por fuerza bruta es más rápido que el cuántico para estos tamaños pequeños de clave. En el límite de hardware de este experimento (22 bits) es posible apreciar que el resultado fue de 333 contra 1057 segundos. Es decir, el clásico fue más rápido.

Esto ocurre principalmente por la complejidad y sobrecarga de procesamiento que requiere la computación cuántica. En estos valores, la ejecución de la función Oráculo y la transformada cuántica de Fourier como parte del circuito implementado en `ibm_marrakesh`, requiere mayor tiempo que la iteración simple del ataque por fuerza bruta clásico.

Sin embargo, haciendo una proyección para un tamaño de clave de 26 bits, siguiendo con el mismo comportamiento polinomial de la cuántica y el exponencial de la clásica, es posible apreciar en la *Imagen 10* que para 24 bits de clave (en proyección), la curva de tiempos por ataque clásico supera la curva de tiempos por ataque cuántico.

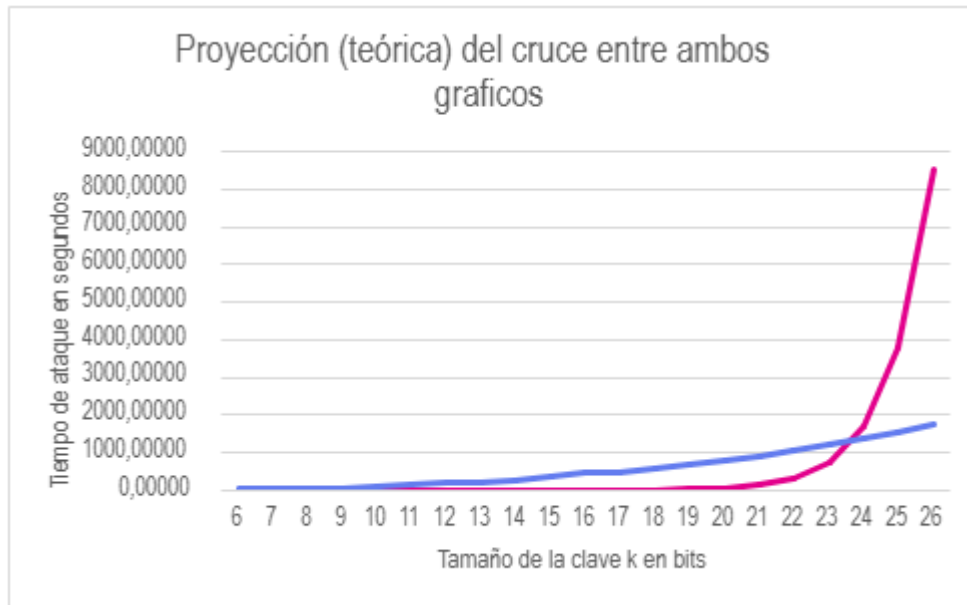


Imagen 10 – Comparativa de tiempos de ambos ataques en proyección – Fuente: elaboración propia a partir de los datos experimentales obtenidos y proyectados

En la *Tabla 4*, se ven estos resultados conjuntos para poder compararlos en números. La proyección de los datos de 23 a 26 bits se consiguió a partir de la información recabada con tamaños menores de clave siguiendo con el comportamiento de ambos esquemas. Para los valores cuánticos $N + 1$, se calcula como $1,15 * N$, ya que $\frac{N+1}{N} = 1,15$ (polinomial). Para los valores clásicos, se calcula $N + 1 = 2,25 * N$ (exponencial).

Tamaño de la clave k en bits	Tiempo en clásico	Tiempo en cuántica
6	0,00260s	21,61106s
7	0,00471s	33,31757s
8	0,00652s	52,22622s
9	0,01551s	72,93733s
10	0,03896s	91,05121s
11	0,05382s	133,16817s
12	0,10471s	192,88850s
13	0,26509s	219,81252s
14	0,51323s	287,54053s
15	1,13776s	349,67285s
16	2,83673s	457,80977s

Tamaño de la clave k en bits	Tiempo en clásico	Tiempo en cuántica
17	7,45398s	487,55161s
18	16,72867s	599,49868s
19	37,48334s	678,25127s
20	70,48966s	807,40971s
21	152,94489s	919,57429s
22	333,10265s	1057,34532s
23	749,48096s	1216,70000s
24	1686,33216s	1382,40000s
25	3794,24735s	1562,50000s
26	8537,05654s	1757,60000s

Tabla 4 – Resultados de la comparativa entre ambos ataques proyectados – Fuente: elaboración propia a partir de los datos experimentales obtenidos y proyectados

La analogía es que, con los valores que se han podido experimentar con claves pequeñas y curvas de juguete, el ataque cuántico mantiene una tasa de crecimiento sostenible mientras aumenta el tamaño de la clave, mientras que la del clásico no. La *Tabla 4* y la *Imagen 10* evidencian este comportamiento.

La verdadera amenaza se aprecia en los resultados proyectados (tal como la teoría indicaba). Si se proyectan ambos tamaños de clave a 256 bits de tamaño, el ataque clásico tardaría $2,72 \times 10^{77}$ años (proyección estimativa). Para tener referencia del orden de magnitudes que esto conlleva, la edad de la Tierra es de aproximadamente $4,54 \times 10^9$ años, o 4,54 mil de millones de años [19]. Absolutamente inviable de resolver, y esto es porque ECC clásicamente es seguro y queda evidenciado aquí que, en una curva real como “secp256k1”, no tendría ningún tipo de sentido intentar realizar el ataque por fuerza bruta clásico.

En contraste, el ataque cuántico implementando el algoritmo de Shor arroja una proyección aproximada de 19,42 días para romper una clave de 256 bits generada con ECC con los resultados obtenidos. Aunque esta diferencia es abismal

respecto a los tiempos clásicos, con el hardware disponible hoy en día no representa una amenaza inmediata.

Es crucial matizar la proyección lineal teórica con las limitaciones físicas reales: tal como demostraron Gidney y Ekerå (2019) [21], para romper criptografía comparable en tiempos razonables, el tiempo real de cómputo en una máquina tolerante a fallas sería significativamente mayor al que sugiere la proyección lineal del presente trabajo.

7.3.2 Complejidad Computacional

Los resultados experimentales visibles en la *Tabla 1* demuestran que el tiempo requerido para romper una clave generada por curvas elípticas mediante métodos clásicos crece a una tasa aproximada de 2,25 bits por cada bit adicional de clave.

El resultado confirma empíricamente que la complejidad del ataque por fuerza bruta clásico exhibe un comportamiento exponencial experimental de $2,25^N$, lo cual es consistente con la clase de complejidad teórica $O(2^N)$ y la teoría de búsqueda en el espacio de claves binarias, en donde el universo se duplica con cada incremento unitario en el tamaño de la clave, es decir $2^{N+1} = 2 * 2^N$.

Sin embargo, al analizar los resultados de la *Tabla 2*, correspondientes a los tiempos requeridos por el método cuántico para cada tamaño de clave en bits, se observa un comportamiento radicalmente distinto al clásico. Los datos cuánticos en este caso se ajustan a una curva polinomial de grado 3 de la forma $O(N^3)$.

7.3.2.1 Cantidad de Qubits

Se suma a este análisis los resultados obtenidos de la cantidad de qubits requeridos para cada tamaño de clave en bits. Como se ve en la *Tabla 3* para romper una clave de 20 bits, son requeridos 140 qubits aproximadamente, no 2^{20} . Esto quiere decir que el requerimiento físico es distinto en la computación cuántica a lo que es en la clásica, ya que la complejidad clásica se dispara exponencialmente, lo que demuestra que el algoritmo de Shor es escalable en tanto al uso de hardware.

7.4 Consideraciones para tener en cuenta

Cabe aclarar a modo de disclaimer que los resultados obtenidos aquí han sido obtenidos en base a los algoritmos ejecutados y desarrollados. Si bien el algoritmo es funcional, futuras iteraciones podrían optimizar la funcionalidad del circuito utilizando técnicas avanzadas para obtener una versión más óptima del algoritmo de Shor implementado en un circuito cuántico. (trabajo a futuro)

Por lo tanto, las diferencias obtenidas en los tiempos podrían variar dado un programa más optimizado en el uso de las funciones de la librería de Qiskit. Tal vez se puede hacer un mejor uso para acelerar la ejecución (variación de tiempos) o reducir la cantidad de qubits necesarios (trabajo a futuro).

Sin embargo, esto no afectará el análisis, dado que igualmente se cumple la hipótesis y la teoría inicial de que, cuánticamente aumenta de manera polinomial el tiempo requerido conforme aumenta el tamaño de las claves en bits.

El objetivo del trabajo era contrastar la naturaleza exponencial clásica frente a la polinomial cuántica. Para fines demostrativos en una prueba de concepto, la fuerza bruta ilustra perfectamente la barrera exponencial. Incluso si se hubiera

utilizado Pollard's Rho, la complejidad clásica seguiría siendo exponencial: $O\left(2^{\frac{N}{2}}\right)$, por lo que la conclusión de que la cuántica es superior a largo plazo se mantiene inalterada.

Por otro lado, al ser una prueba de concepto, las claves que se rompieron estuvieron en el rango de entre 6 y 22 bits de tamaño, mientras que la curva estándar utiliza 256 bits. Si bien los resultados validan la teoría, no se ha efectuado un ataque a una clave de criptografía por curvas elípticas real actual.

Asimismo, es fundamental precisar el alcance de la implementación de la función Oráculo dentro del circuito cuántico desarrollado. Para este trabajo, se optó por un enfoque híbrido en el cual la aritmética modular se gestiona mediante tablas de verdad pre-calculadas, en lugar de realizar el cómputo aritmético completo dentro de la *QPU*. Esta decisión metodológica responde a las limitaciones físicas del hardware actual, dado que la profundidad de circuito requerida para una implementación aritmética pura excedería los tiempos de coherencia de los qubits, provocando que el ruido acumulado destruya la información cuántica antes de la medición.

No obstante, esta aproximación se valida como una prueba de concepto efectiva, ya que permite aislar y demostrar el funcionamiento del núcleo real del algoritmo de Shor: la subrutina de búsqueda de periodo (*Period Finding*) y la Transformada Cuántica de Fourier (*QFT*). De esta manera, se logra verificar la capacidad de la computación cuántica para hallar patrones periódicos.

Proyectando los resultados en cantidad de qubits necesarios para romper una clave de 256 bits de ECC con computación cuántica, se requerirían 1792 qubits

aproximadamente. Esto quiere decir que, la barrera actual en el mundo real es la cantidad de qubits disponibles en hardware.

Otro punto para tener en cuenta es que los resultados de la *Tabla 2* representan únicamente el tiempo puro de cómputo cuántico *QPU*. A este hay que sumarle la pre-ejecución clásica, la transpilación, la cola de espera, la latencia de red y el tiempo requerido por el post-procesado clásico para recuperar k a partir del periodo devuelto por el algoritmo. Esto quiere decir que los 1057 segundos aproximados requeridos para romper una clave de 22 bits no significan realmente 1057 segundos, sino que el tiempo es todavía mayor, pero no se tuvo en cuenta en el presente trabajo.

Otra barrera en el mundo real que se debe tener en cuenta es el costo de implementación en la computación cuántica. Cada segundo extra de los 10 minutos gratuitos que brinda IBM cuesta 1,60 US\$. Lo cual es un costo realmente elevado si se quiere escalar el ataque a mayor cantidad de bits en el tamaño de la clave. En este trabajo se ha utilizado para el experimento el hardware real brindado por IBM de manera gratuita. En otro desarrollo, puede caber la posibilidad de pagar por el servicio extra y contar con mayores cantidades de qubits disponibles para realizar pruebas (trabajo a futuro).

Por último, es necesario resaltar el uso de la inteligencia artificial (IA) como herramienta de desarrollo de código y guía de uso de las librerías utilizadas de Qiskit. Aunque se haya utilizado la IA como colaboración, los programas requirieron análisis, pruebas, correcciones y validaciones. Por ejemplo, al no contar con resultados de otros experimentos cuánticos en el mismo campo de estudio, no existió referencia de la cual se pueda extraer información de los tiempos necesarios, por lo que se tuvo que validar y revalidar la información teniendo en cuenta que los tiempos

en cuántica (para este experimento entre 6 y 22 bits) eran mayores que los tiempos en el ataque por fuerza bruta clásico.

8 Conclusión

Se logro desarrollar satisfactoriamente un circuito cuántico ejecutable que aplica la lógica del algoritmo de Shor y permite resolver el logaritmo discreto elíptico independientemente de la clave.

Lo resultados que se obtuvieron de la medición con las métricas se lograron gracias al enfoque hibrido adoptado (clásico + cuántico), decisión necesaria para cumplir el objetivo, ya que un enfoque puramente cuántico habría fallado por ruido y limitaciones del hardware.

En función a los resultados obtenidos (ver *sección 7*) y su posterior análisis (ver *sección 7.1.3*) es posible afirmar que se ha logrado implementar el circuito de Shor en una computadora cuántica real.

Los resultados demostraron que, con el hardware disponible, el ataque cuántico es más lento que el clásico para claves pequeñas, pero en el análisis general, con el incremento del tamaño de clave, se replica la teoría de Shor en la práctica donde la complejidad del ataque por fuerza bruta clásico crece exponencialmente ($2,25^N$), mientras que el del ataque cuántico crece de manera polinomial (N^3).

Operar con instancias reducidas (hasta 22 bits) para obtener los resultados permitió realizar la proyección hacia tamaños de clave mayores y lograr el punto teórico de intersección entre ambos esquemas (24 bits en la proyección). Este fue el puente fundamental para extender los resultados a claves reales de 256 bits y obtener un orden estimativo de magnitudes.

Se llegó a ejecutar el experimento, hasta alcanzar el máximo posible, ya que se encuentra con la limitación del hardware cuántico, en cuanto se disponga de

mayores cantidades de qubits para utilizar, se podrán realizar experimentos a mayor escala y buscar llegar al punto de intersección entre los tiempos requeridos del ataque clásico y cuántico, resultado viable debido a la escalabilidad en uso de recursos que se logró demostrar.

Para finalizar, los resultados confirman la postura del NIST haciendo énfasis en el desarrollo de la denominada criptografía post-cuántica. Los resultados indican que, hoy en día, la criptografía de curva elíptica está a salvo, pero la escalabilidad polinomial de cómputo que ofrece la computación cuántica comprende una amenaza fundamental en el mundo de la criptografía a futuro. El problema de resolver el logaritmo discreto elíptico ya está resuelto en la teoría, es cuestión de avanzar en el hardware para cambiar completamente el paradigma.

9 Futuras Discusiones

Si bien el presente trabajo ha logrado validar empíricamente la diferencia entre complejidades de un ataque clásico y uno cuántico para resolver el problema del logaritmo discreto elíptico, la investigación abre nuevas preguntas a futuro.

La tarea más inmediata para un futuro es validar empíricamente el punto de cruce entre las curvas de tiempos del ataque clásico y cuántico. En este trabajo se hizo una proyección teórica a partir de los resultados obtenidos. En cuanto se pueda superar la limitación de qubits, será posible ejecutar las pruebas para cuando el tamaño de la clave supere los 22 bits y registrar el momento en el que se interceptan ambas curvas.

Otro punto para tener en cuenta es el de medir el tiempo real de ejecución cuántico, no solo el de QPU, lo cual fue objetivo del presente trabajo. Además, se invita a futuros investigadores a utilizar las nuevas generaciones de hardware disponibles y medir la calidad de respuesta que presentan, no solo la cantidad de qubits que disponen.

Para finalizar, los desarrollos de código siempre pueden ser optimizados conforme se lancen nuevas versiones de Qiskit o de hardware. Es probable que pueda utilizarse menor cantidad de qubits y lograr ataques más viables en hardware más pequeño, y con menores tiempos.

10 Bibliografía

- [1] W. Diffie y M. Hellman, “New directions in cryptography,” *IEEE Trans. Inf. Theory*, vol. 22, no. 6, pp. 644–654, nov. 1976, doi: 10.1109/TIT.1976.1055638.
- [2] F. Rojo Amadeo, “Computación cuántica: Análisis y ejecución de algoritmos cuánticos,” Tesis de Licenciatura, Pontificia Univ. Católica Argentina, Buenos Aires, Argentina, 2020.
- [3] Kolokium, “Criptografía de Curva Elíptica,” *Kolokium*, 30 de mayo de 2023. [En línea]. Disponible en: <https://kolokium.com/blog/criptografia-de-curva-eliptica/>. [Accedido: 24-ago-2025].
- [4] J. Proos y C. Zalka, “Shor’s discrete logarithm quantum algorithm for elliptic curves,” *arXiv preprint arXiv:quant-ph/0301141*, 2003.
- [5] R. Bavdekar, E. Jayant Chopde, A. Agrawal, A. Bhatia y K. Tiwari, “Post Quantum Cryptography: A Review of Techniques, Challenges and Standardizations,” en *2023 International Conference on Information Networking (ICOIN)*, ene. 2023, pp. 146–151. doi: 10.1109/ICOIN56518.2023.10048976.
- [6] S. B. Hegde, A. Jamuar y R. Kulkarni, “Post Quantum Implications on Private and Public Key Cryptography,” en *2023 International Conference on Smart Systems for applications in Electrical Sciences (ICSSES)*, jul. 2023, pp. 1–6. doi: 10.1109/ICSSES58299.2023.10199503.
- [7] IBM, “Qiskit guides,” *IBM Quantum Documentation*. [En línea]. Disponible en: <https://quantum.cloud.ibm.com/docs/es/guides>. [Accedido: 05-nov-2025].

- [8] IBM, “Qiskit,” *Qiskit: An Open-source Framework for Quantum Computing*. [En línea]. Disponible en: <https://github.com/Qiskit/qiskit>. [Accedido: 05-nov-2025].
- [9] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, oct. 1997, doi: 10.1137/S0097539795293172.
- [10] D. S. C. Putranto, R. W. Wardhani, J. Ji, y H. Kim, “A Deep Inside Quantum Technology Industry Trends and Future Implications,” *IEEE Access*, vol. 12, pp. 115776–115801, ago. 2024, doi: 10.1109/ACCESS.2024.3444779.
- [11] N. Koblitz, “Elliptic curve cryptosystems,” *Math. Comput.*, vol. 48, no. 177, pp. 203–209, 1987, doi: 10.1090/S0025-5718-1987-0866109-5.
- [12] Grupo GeoGebra, “Calculadora gráfica de GeoGebra.” [En línea]. Disponible en: <https://www.geogebra.org/calculator>. [Accedido: 05-nov-2025].
- [13] A. J. Menezes, P. C. van Oorschot y S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL, USA: CRC Press, 1996. [En línea]. Disponible en: <https://cacr.uwaterloo.ca/hac/>.
- [14] L. Chen *et al.*, “Recommendations for Discrete Logarithm-based Cryptography,” National Institute of Standards and Technology, Gaithersburg, MD, NIST Special Publication 800-186, feb. 2023. doi: 10.6028/NIST.SP.800-186.
- [15] D. R. L. Brown, “SEC 2: Recommended Elliptic Curve Domain Parameters,” Standards for Efficient Cryptography Group (SECG), Tech. Rep., ene. 2010. [En línea]. Disponible en: <https://www.secg.org/sec2-v2.pdf>.
- [16] Research Institute CODE, “CrypTool,” 1998. [Software]. Disponible en: <https://www.cryptool.org>.
- [17] S. M. Hosseini y H. Pilaram, “A comprehensive review of post-quantum cryptography: Challenges and advances,” *International Journal of Data and*

- Network Science*, vol. 9, no. 2, pp. 267–288, 2025. doi: 10.5267/j.ijdns.2024.12.003.
- [18] M. A. Nielsen y I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary ed. Cambridge, U.K.: Cambridge Univ. Press, 2010, pp. 216–242.
- [19] G. B. Dalrymple, *The Age of the Earth*. London, U.K.: Palgrave Macmillan, 1991. doi: 10.1007/978-1-349-25249-7_3.
- [20] M. Roetteler, M. Naehrig, K. M. Svore y K. Lauter, “Quantum resource estimates for computing elliptic curve discrete logarithms,” en *Advances in Cryptology – ASIACRYPT 2017*, Cham, Suiza: Springer, 2017, pp. 241–270, doi: 10.1007/978-3-319-70697-9_9.
- [21] C. Gidney y M. Ekerå, “How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits,” *Quantum*, vol. 5, p. 433, abr. 2021, doi: 10.22331/q-2021-04-15-433.
- [22] T. Häner, M. Roetteler y K. M. Svore, “Factoring using $2n+2$ qubits with Toffoli based modular multiplication,” *Quantum Science and Technology*, vol. 5, no. 2, p. 025003, 2020, doi: 10.1088/2058-9565/ab6750.

11 Anexos

11.1 Anexo 1: Qiskit Runtime Service

Esta librería es la que utilizamos para ejecutar el circuito cuántico en las computadoras cuánticas reales de IBM de manera rápida y eficiente. Su principal objetivo es reducir la latencia. El problema que resolvió fue el ida y vuelta que se generaba entre la computadora cuántica (QPU) y la PC de origen de la comunicación. El cambio de modelo que propone es empaquetar en la PC de origen el programa completo, y eso enviarlo a IBM Cloud. Este programa se ejecuta en una CPU clásica que se comunica directamente con la QPU.

Esto resuelve el cálculo de ida y vuelta, anticipando el problema de colas infinitas y latencia de red. Además, colabora en la mitigación de errores, el ruido se disminuye y las respuestas son más precisas.

11.2 Anexo 2: OPENQASM 2.0

Este es el código *OPENQASM 2.0* que ejecutamos en el Composer de la plataforma de IBM Quantum para obtener una representación de baja escala y en modo de caja negra del circuito de nuestra implementación de Shor. Al mismo se lo puede acceder mediante el siguiente enlace de repositorio de Github:

https://github.com/Isoldios/TF_Isoldi/blob/main/OPENQASM-2_0.txt

11.3 Anexo 3: Software ejecutable (lenguaje Python)

11.3.1 `busqueda_curva.ipynb`

Este archivo de código Jupyter Notebook en Python corresponde a la búsqueda del máximo orden de la curva r , y un punto que satisfaga esa condición

para utilizarlo como generador G . Al mismo se lo puede acceder mediante el siguiente enlace de repositorio de Github:

https://github.com/Isoldios/TF_Isoldi/blob/main/busqueda_curva.ipynb

11.3.2shor_breaking_ecc.ipynb

Este archivo de código Jupyter Notebook en Python corresponde al corazón del presente trabajo. El mismo implementa el algoritmo de Shor expresado en un circuito cuántico ejecutable. Se lo puede acceder mediante el siguiente enlace de repositorio de Github:

https://github.com/Isoldios/TF_Isoldi/blob/main/shor_breaking_ecc.ipynb

11.3.3ataque_ECC_fuerza_bruta_clasico.ipynb

Este código implementa el ataque por fuerza bruta clásico a ECC según los parámetros que le asignemos. El mismo mide el tiempo requerido de recorrer y probar todas las claves posibles, suponiendo que, en el peor de los casos, ese debería ser el comportamiento. Al mismo se lo puede acceder mediante el siguiente enlace de repositorio de Github:

https://github.com/Isoldios/TF_Isoldi/blob/main/ataque_ECC_fuerza_bruta_clasico.ipynb